

Contents

Adding a Sensor	1
Can This Sensor Be Added Without a Rebuild?	1
Step-by-Step: Adding a Sensor	1
Testing a Draft Before You Publish	2
Built-In Templates	2
Constraints and Locks	3
Appendix A: Profile JSON Schema	3
Appendix B: Built-In Profile Examples	5
Xiaomi LYWSD03MMC Temperature and Humidity	5
Kestrel Drop 2 Temperature and Humidity	5
Inkbird IBS-TH1, IBS-TH1 Mini, and IBS-TH1 Plus Temperature and Humidity	6
PC-60FW Pulse Oximeter	7
Flower Care Plant Sensor	8
RuuviTag 4-in-1 (Temperature, Humidity, Pressure, Movement)	9

Adding a Sensor

This guide explains how GeoSenEsm sensor integrations are configured, how to test a new one before it goes live, and where the boundary is between a configuration-only change and a mobile application rebuild. Detailed schema/example reference is pushed to the [Appendix](#); read the sections below first.

Can This Sensor Be Added Without a Rebuild?

Configuration only (no app rebuild) works when the sensor fits GeoSenEsm’s BLE profile engine:

1. Bluetooth Low Energy, discoverable by advertised name, name prefix, service UUID, manufacturer data, or Xiaomi MiBeacon service data.
2. Read sequence expressible as `read`, `notification`, `indication`, `write`, and `delay` steps.
3. Payload decodable with scalar types `uint8`, `int8`, `uint16`, `int16`, `uint32`, `int32`, `float32`, or `sfloat16`.
4. Frame validation needs at most a fixed length, optional prefix, and `crc8_maxim` (or no checksum).
5. Output values map to raw parameter codes (`temperature`, `humidity`, `spo2`, `pulse_rate`, `light`, `moisture`, `conductivity`, `opening`, or a new code of your choosing).

A rebuild is required instead when the sensor needs: a non-BLE transport (Classic Bluetooth, USB, NFC, Wi-Fi, vendor SDK); a proprietary handshake beyond `write/delay`; an advertisement decoder other than the whitelisted `xiaomi_mibeacon_v4_v5` (**unencrypted only** — see [Constraints](#)) or `ruuvi_data_format_5`; a new checksum/compression/multi-packet state machine; new mobile permissions, background behavior, or UI; or a native/coded adapter instead of `integration_mode = 'profile'`.

Step-by-Step: Adding a Sensor

1. Create the sensor type on **Settings** → **Add new sensor type** with a stable lowercase code and `integration_mode = 'profile'` (POST `/api/sensorprofiles/types`) — or materialize a [built-in template](#) instead via POST `/api/sensorprofiles/templates/{code}/install`, which does steps 1–4 for you.
2. Declare the sensor type’s raw parameter codes (POST `/api/sensorprofiles/types/{sensorTypeId}/parameters`) — these must match the “parameter” values your profile JSON will use.
3. Promote each raw parameter into “used sensor data” (POST `.../parameters/{id}/use`): link to an existing `sensor_parameter_definition` to add it as an extra source, or create a new one. A **used** parameter’s identity is the (name, unit) pair — a different unit for the same concept (e.g. Flower Care’s lux-based `light` vs. a boolean `light_detected`) needs its own used parameter.

4. Author the profile JSON (see [Appendix](#)), create a draft `sensor_gatt_profile` revision, and validate it — the validator checks structure and re-decodes every `goldenPackets` entry you provide.
5. **Before publishing**, test the draft against a real device from the browser — see [Testing a Draft](#).
6. Publish the profile revision, then enable the sensor type and set its connection timeout on **Settings** → **Integrations**.
7. Assign the source or physical device to respondents. A pre-registered MAC (`sensor_mac.sensor_mac`) is optional — a sensor matched purely by discovery (name/service/product ID) can be assigned without knowing the physical unit in advance.
8. Ask respondents to refresh the mobile app (`/api/surveysettings/sensordata/mobile` syncs the new setup) and complete a full test survey.
9. Confirm SQL has one `sensor_data` row with matching `sensor_data_parameter_value` rows, and the Mongo response document has a matching `sensorData` entry (`source + values`).

No app rebuild is needed for any of this — the backend just sends the active profile to the existing BLE engine on the phone.

Testing a Draft Before You Publish

Three layers of testing exist, cheapest first:

- **Golden packets** (static, no hardware): include one or more `goldenPackets` entries in the profile JSON; the backend validator decodes each one server-side and rejects the profile if the result doesn't match `expected`. Works for both `gatt_sequence` and `ble_advertisement` (`ruuvi_data_format_5` only — MiBeacon can't be re-decoded server-side, see [Appendix](#)).
- **Live browser test** (real hardware, no phone needed): on the sensor profile draft page, the “**Test with a real device**” panel uses the Web Bluetooth API to connect straight from the browser and run the draft's actual `write/delay/acquire` steps against it, showing raw hex and decoded values per step before you touch a phone or publish anything. Requirements and limits:
 - Chrome or Edge on desktop with Bluetooth — unsupported browsers (Firefox, Safari, and Web Bluetooth generally on iOS) show a disclaimer instead of the panel.
 - **`gatt_sequence` profiles only.** Web Bluetooth can connect and read/write characteristics but can't passively scan raw advertisements the way the mobile app does, so `ble_advertisement`/MiBeacon profiles can't be exercised this way — verify those with a phone-based BLE scanner (e.g. nRF Connect) instead.
 - The device picker uses `acceptAllDevices` with every `serviceUuid` referenced anywhere in the draft pre-declared as `optionalServices` (Web Bluetooth requires this before it grants access, even when picking by name).
- **End-to-end mobile test:** refresh the app on a real phone and complete a full survey (step 8–9 above) — the only way to confirm discovery, timeouts, and persistence all agree.

Built-In Templates

Six pre-built sensors — Xiaomi LYWSD03MMC, Kestrel Drop 2, Inkbird IBS-TH1 (/Mini/Plus), PC-60FW, Flower Care, Ruuvi Tag 4-in-1 — exist as code-defined templates in `SensorProfileTemplateCatalog` (not Flyway seed rows; the `sensor_type` catalog ships empty). Activating one via **Settings** → **Integrations** → **Available templates** (or `POST/api/sensorprofiles/templates/{templateCode}/install`) creates the `sensor_type`, its `sensor_type_setting`, one `sensor_type_parameter` row per mapped parameter (auto-promoted — no manual “use” step needed), and a published profile revision, then surfaces the new type at the top of the active list to review its timeout. Installing is idempotent per template code; a used parameter a template needs is created once and reused by every later install (built-in or custom) that needs the same code. Full JSON for each template is in the [Appendix](#).

`none` and `manual` are two reserved `sensor_type` rows, never templates and never offered in “pick a sensor” menus — `none` means “no sensor data,” `manual` is the always-present fallback wired unconditionally onto every used parameter the moment it's created (`SensorTypeParameterServiceImpl.ensureManualSource`), so a respondent can always be prompted to enter a value by hand.

Constraints and Locks

- **Publish lock.** Once `InitialSurveyService.isPublished()` is `true`, all sensor-setup mutations (creating/enabling sensor types, raw-parameter catalog changes, used-parameter edits, sensor data mode, template installs, GATT profile draft/publish/rollback) are rejected with 400. The admin panel disables the relevant controls once published. Respondent sensor *assignments* are the one exception — always writable via `PUT/api/sensormac/{sensorId}/respondent`, since who has which physical sensor keeps changing through a live study.
- **Collected-data lock**, independent of publish state: once any `sensor_data` row exists (`sensorDataRepository.count()>0`), saving **Settings** → **Integrations** (which enables/disables sensor types and changes the sensor data mode) and deleting a sensor type (`DELETE/api/sensorprofiles/types/{sensorTypeId}`) are both blocked outright — either operation could otherwise orphan or destroy collected readings. `manual/none` can never be deleted.
- **Disabling vs. deleting a sensor type.** Disabling (unchecking it on Integrations) un wires every parameter source it fed — a used parameter left with zero sources is deleted, except its guaranteed `manual` source, which is never removed this way. Re-enabling does **not** auto-restore old links; wire each raw parameter back by hand. Deleting the type outright removes the `sensor_type` row itself (its `settings/MAC/assignment` rows are cleaned up first; raw parameters and profiles cascade).
- **No per-parameter active/inactive toggle.** A used parameter is either on the list or removed via `DELETE/api/surveysettings/sensordata/parameters/{id}`, which itself returns 409 if readings have already been collected for it.
- **No priority between sources.** Every source that reports a value for a parameter is saved as its own independent reading — there's no “winning” source. A parameter is only offered for manual entry if none of its sources produced a value on a fresh connection attempt at survey end.
- **No secrets support.** GeoSenEsm cannot decode a device bound/encrypted with a per-device secret (e.g. a Xiaomi MiBeacon `bind_key`) — that machinery (`sensor_device_secret`, the admin `bind-key` field, mobile decryption) was removed entirely. Only unbound/unencrypted MiBeacon advertisements work.

Appendix A: Profile JSON Schema

A published profile lives in `sensor_gatt_profile.spec_json`, describing device discovery, BLE operations, frame validation, and byte-to-parameter decoding. `sensor_type_parameter` is a sensor type's own **raw** catalog (what it can produce — scoped per type, so the same raw code can repeat across types); `sensor_parameter_definition` is the globally-unique **used** list actually collected and exported. Readings are stored as a *list*: one `SensorData` row per connected sensor type per submission (a respondent can carry more than one sensor at once), mirrored in the Mongo response document's `sensorData` array and the `GET /api/sensordata` export.

Minimal GATT profile:

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "Device name",
    "serviceUuid": "00000000-0000-1000-8000-00805f9b34fb"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "00000000-0000-1000-8000-00805f9b34fb",
      "characteristicUuid": "00000001-0000-1000-8000-00805f9b34fb",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
    }
  ]
}
```

```

    "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
    "assertions": [],
    "decoders": [
      {
        "parameter": "temperature",
        "type": "uint16",
        "offset": 0,
        "endian": "little",
        "scale": 0.01,
        "add": 0,
        "min": -40,
        "max": 125
      }
    ]
  },
  "goldenPackets": [
    {
      "characteristicUuid": "00000001-0000-1000-8000-00805f9b34fb",
      "packetHex": "340C2D",
      "expected": { "temperature": 31.24 }
    }
  ]
}

```

Minimal advertisement profile (TLV/MiBeacon shape):

```

{
  "schemaVersion": 1,
  "transport": "ble_advertisement",
  "advertisement": {
    "decoderId": "xiaomi_mibeacon_v4_v5",
    "matcher": { "productId": 2443 },
    "objects": [
      {
        "objectId": "0x1019",
        "parameter": "opening",
        "type": "uint8",
        "values": { "0": "open", "1": "closed", "2": "left_open" }
      }
    ]
  },
  "goldenPackets": [
    {
      "advertisementHex": "50308B09000000000000000019100100",
      "expected": { "opening": "open" }
    }
  ]
}

```

An object's **type** may be `uint8`, `int8`, `uint16`, `int16`, or `bool`, and can optionally carry a **scale** the same way a GATT decoder does. `discovery.serviceUuid` is optional here too — some devices are matched purely by advertised name. Stock firmware only encrypts the advertisement once bound to a Mi Home account; unbound devices broadcast the object payload in the clear, no key involved. A device that ships already bound, or that a respondent has ever paired with the Xiaomi Home app, cannot be integrated via

this transport.

The advertisement transport also supports a fixed-offset shape for devices with no TLV framing at all (Ruuvi's sensors): `decoderId:"ruuvi_data_format_5"`, matched via `matcher.manufacturerId` instead of a Xiaomi `productId`, described with `decoders` (the same offset/type/endian/scale/add/min/max shape a `gatt_sequence` read uses). Unlike MiBeacon, a fixed-offset decoder's golden packets are actually decoded and cross-checked server-side. `discovery` is never populated for either advertisement shape — the device is identified entirely from the advertisement payload (service UUID or manufacturer ID).

Appendix B: Built-In Profile Examples

Shortened to focus on the connection definition.

Xiaomi LYWSD03MMC Temperature and Humidity

Simple read-based GATT profile: discover by name alone, connect to its own GATT service, read one characteristic, decode temperature and humidity from three bytes. Real units broadcast an encrypted MiBeacon advertisement that's unreadable without a bind key, so this deliberately reads the plaintext GATT characteristic directly instead of using `ble_advertisement`.

```
{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "LYWSD03MMC"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "ebe0ccb0-7a0a-4b0c-8a1a-6ff2997da3a6",
      "characteristicUuid": "ebe0ccc1-7a0a-4b0c-8a1a-6ff2997da3a6",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
      "assertions": [],
      "decoders": [
        { "parameter": "temperature", "type": "uint16", "offset": 0, "endian": "little",
          ↪ "scale": 0.01, "add": 0, "min": -40, "max": 125 },
        { "parameter": "humidity", "type": "uint8", "offset": 2, "endian": "little",
          ↪ "scale": 1, "add": 0, "min": 0, "max": 100 }
      ]
    }
  ],
  "goldenPackets": [
    {
      "characteristicUuid": "ebe0ccc1-7a0a-4b0c-8a1a-6ff2997da3a6",
      "packetHex": "66082D",
      "expected": { "temperature": 21.5, "humidity": 45 }
    }
  ]
}
```

Kestrel Drop 2 Temperature and Humidity

Two separate readable characteristics. Discovers a respondent-assigned device by interpolating the physical sensor identifier into D2 - `{sensorId}`, while also accepting the shared Kestrel service UUID.

```

{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "D2 - {sensorId}",
    "namePrefix": "D2 - ",
    "serviceUuid": "12630000-cc25-497d-9854-9b6c02c77054"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "12630000-cc25-497d-9854-9b6c02c77054",
      "characteristicUuid": "12630001-cc25-497d-9854-9b6c02c77054",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
      "assertions": [{ "offset": 0, "equals": 7 }],
      "decoders": [
        { "parameter": "temperature", "type": "uint16", "offset": 1, "endian": "little",
          ↪ "scale": 0.01, "add": 0, "min": -40, "max": 125 }
      ]
    },
    {
      "kind": "acquire",
      "serviceUuid": "12630000-cc25-497d-9854-9b6c02c77054",
      "characteristicUuid": "12630002-cc25-497d-9854-9b6c02c77054",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 3, "prefixHex": "", "checksum": "none" },
      "assertions": [{ "offset": 0, "equals": 7 }],
      "decoders": [
        { "parameter": "humidity", "type": "uint16", "offset": 1, "endian": "little",
          ↪ "scale": 0.01, "add": 0, "min": 0, "max": 100 }
      ]
    }
  ],
  "goldenPackets": [
    {
      "characteristicUuid": "12630001-cc25-497d-9854-9b6c02c77054",
      "packetHex": "073408",
      "expected": { "temperature": 21 }
    },
    {
      "characteristicUuid": "12630002-cc25-497d-9854-9b6c02c77054",
      "packetHex": "079411",
      "expected": { "humidity": 45 }
    }
  ]
}

```

Inkbird IBS-TH1, IBS-TH1 Mini, and IBS-TH1 Plus Temperature and Humidity

Advertises as `sps` with the custom `FFF0` service. The live-data characteristic `FFF2` holds temperature (hundredths of a degree C) then relative humidity (hundredths of a percent).

```

{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "sps",
    "serviceUuid": "0000fff0-0000-1000-8000-00805f9b34fb"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "0000fff0-0000-1000-8000-00805f9b34fb",
      "characteristicUuid": "0000fff2-0000-1000-8000-00805f9b34fb",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 7, "prefixHex": "", "checksum": "none" },
      "assertions": [],
      "decoders": [
        { "parameter": "temperature", "type": "int16", "offset": 0, "endian": "little",
          ↪ "scale": 0.01, "add": 0, "min": -40, "max": 125 },
        { "parameter": "humidity", "type": "uint16", "offset": 2, "endian": "little",
          ↪ "scale": 0.01, "add": 0, "min": 0, "max": 100 }
      ]
    }
  ],
  "goldenPackets": [
    {
      "characteristicUuid": "0000fff2-0000-1000-8000-00805f9b34fb",
      "packetHex": "B107C117000762",
      "expected": { "temperature": 19.69, "humidity": 60.81 }
    }
  ]
}

```

PC-60FW Pulse Oximeter

Uses notifications. Validates the frame prefix and `crc8_maxim` checksum, then maps packet bytes to SpO2, pulse rate, and perfusion index. Discovery matches a name prefix (not the exact model string) since other PC-60-family units share the service but differ slightly in advertised name.

```

{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "namePrefix": "PC-60",
    "serviceUuid": "6e400001-b5a3-f393-e0a9-e50e24dcca9e"
  },
  "operations": [
    {
      "kind": "acquire",
      "serviceUuid": "6e400001-b5a3-f393-e0a9-e50e24dcca9e",
      "characteristicUuid": "6e400003-b5a3-f393-e0a9-e50e24dcca9e",
      "acquisition": { "mode": "notification", "timeoutMs": 30000, "maxPackets": 100 },
      "frame": { "length": 12, "prefixHex": "AA550F0801", "checksum": "crc8_maxim" },
      "assertions": [],
      "decoders": [

```

```

    { "parameter": "spo2", "type": "uint8", "offset": 5, "endian": "little", "scale":
      ↪ 1, "add": 0, "min": 0, "max": 100 },
    { "parameter": "pulse_rate", "type": "uint8", "offset": 6, "endian": "little",
      ↪ "scale": 1, "add": 0, "min": 0, "max": 255 },
    { "parameter": "perfusion_index", "type": "uint8", "offset": 8, "endian":
      ↪ "little", "scale": 0.1, "add": 0, "min": 0, "max": 25 }
  ]
}
],
"goldenPackets": [
{
  "characteristicUuid": "6e400003-b5a3-f393-e0a9-e50e24cca9e",
  "packetHex": "AA550F08016248000F000079",
  "expected": { "spo2": 98, "pulse_rate": 72, "perfusion_index": 1.5 }
}
]
}

```

Flower Care Plant Sensor

Multi-step sequence: writes A01F, waits for the device to prepare data, then reads one characteristic and maps four values. Discovery matches Xiaomi's shared MiBeacon service (**fe95**) plus the exact name to disambiguate — the sensor-data service used in **operations** is only visible after connecting, so it can't be used for discovery.

```

{
  "schemaVersion": 1,
  "transport": "gatt_sequence",
  "discovery": {
    "nameExact": "Flower care",
    "serviceUuid": "0000fe95-0000-1000-8000-00805f9b34fb"
  },
  "operations": [
    {
      "kind": "write",
      "serviceUuid": "00001204-0000-1000-8000-00805f9b34fb",
      "characteristicUuid": "00001a00-0000-1000-8000-00805f9b34fb",
      "payloadHex": "A01F",
      "timeoutMs": 5000
    },
    { "kind": "delay", "durationMs": 750 },
    {
      "kind": "acquire",
      "serviceUuid": "00001204-0000-1000-8000-00805f9b34fb",
      "characteristicUuid": "00001a01-0000-1000-8000-00805f9b34fb",
      "acquisition": { "mode": "read", "timeoutMs": 10000, "maxPackets": 1 },
      "frame": { "length": 10, "prefixHex": "", "checksum": "none" },
      "assertions": [],
      "decoders": [
        { "parameter": "temperature", "type": "int16", "offset": 0, "endian": "little",
          ↪ "scale": 0.1, "add": 0, "min": -40, "max": 125 },
        { "parameter": "light", "type": "uint32", "offset": 3, "endian": "little",
          ↪ "scale": 1, "add": 0, "min": 0, "max": 4294967295 },
        { "parameter": "moisture", "type": "uint8", "offset": 7, "endian": "little",
          ↪ "scale": 1, "add": 0, "min": 0, "max": 100 },

```



```

    { "parameter": "conductivity", "type": "uint16", "offset": 8, "endian": "little",
      ↪ "scale": 1, "add": 0, "min": 0, "max": 65535 }
  ]
},
"goldenPackets": [
  {
    "characteristicUuid": "00001a01-0000-1000-8000-00805f9b34fb",
    "packetHex": "D70000393000002D5E01",
    "expected": { "temperature": 21.5, "light": 12345, "moisture": 45, "conductivity":
      ↪ 350 }
  }
]
}

```

RuuviTag 4-in-1 (Temperature, Humidity, Pressure, Movement)

Broadcasts a fixed, unencrypted 24-byte struct as manufacturer-specific data (company id 0x0499, big-endian fields) per Ruuvi's own published Data Format 5 (RAWv2) spec, rather than a TLV-framed MiBeacon payload. It also carries acceleration X/Y/Z, a packed battery-voltage/TX-power field, a sequence number, and its MAC — none of which this template decodes. **pressure** is decoded straight to hPa (**scale**: 0.01, **add**: 500) rather than Ruuvi's native whole-Pascal units — a unit fix, not a precision loss, since 1 Pa maps exactly to 2 decimal digits in hPa.

```

{
  "schemaVersion": 1,
  "transport": "ble_advertisement",
  "advertisement": {
    "decoderId": "ruuvi_data_format_5",
    "matcher": { "manufacturerId": 1177 },
    "decoders": [
      { "parameter": "temperature", "type": "int16", "offset": 1, "endian": "big",
        ↪ "scale": 0.005, "add": 0, "min": -163.835, "max": 163.835 },
      { "parameter": "humidity", "type": "uint16", "offset": 3, "endian": "big", "scale":
        ↪ 0.0025, "add": 0, "min": 0, "max": 163.835 },
      { "parameter": "pressure", "type": "uint16", "offset": 5, "endian": "big", "scale":
        ↪ 0.01, "add": 500, "min": 500, "max": 1155.35 },
      { "parameter": "movement", "type": "uint8", "offset": 15, "endian": "big", "scale":
        ↪ 1, "add": 0, "min": 0, "max": 254 }
    ]
  },
  "goldenPackets": [
    {
      "advertisementHex": "0512FC5394C37C0004FFFC040CAC364200CDCBB8334C884F",
      "expected": { "temperature": 24.3, "humidity": 53.49, "pressure": 1000.44,
        ↪ "movement": 66 }
    }
  ]
}

```

The golden packet above is Ruuvi's own published RAWv2 test vector, not a synthetic example — since this decoder is fully decoded and cross-checked server-side, it doubles as a real regression test.